

Process Termination

Definition

Process termination is the point at which a running process stops execution and leaves the system. A process may terminate **voluntarily** after finishing its work or **involuntarily** because of an error or an external kill request.

1. Four Main Reasons for Termination

Termination Cases

A process usually terminates for one of four reasons:

1. **Normal exit** (voluntary)
2. **Error exit** (voluntary)
3. **Fatal error** (involuntary)
4. **Killed by another process** (involuntary)

Reason	Type	Meaning
Normal exit	Voluntary	The process finishes its intended job and asks the OS to terminate it.
Error exit	Voluntary	The process detects a problem itself and exits cleanly.
Fatal error	Involuntary	The OS stops the process because of an illegal or invalid action.
Killed by another process	Involuntary	Another authorized process explicitly requests its termination.

2. Normal Exit

Main Idea

Most processes terminate normally because they have completed the work they were created to do.

Examples

- A compiler terminates after compiling the source program.
- A word processor or browser terminates when the user chooses the close or exit option.

System Calls

To terminate normally, a process typically uses a system call:

- UNIX: `exit`
- Windows: `ExitProcess`

Clean Shutdown

Interactive programs often remove temporary files, release resources, and then terminate cleanly when the user asks them to close.

3. Error Exit

Meaning

An **error exit** happens when the process itself detects that it cannot continue correctly and chooses to terminate.

Compiler Example

Suppose a user enters:

```
cc foo.c
```

If `foo.c` does not exist, the compiler reports the problem and exits.

Interactive Programs

Screen-oriented interactive applications often handle bad input differently. Instead of terminating, they usually display a dialog box and ask the user to try again.

4. Fatal Error

Involuntary Termination

A **fatal error** occurs when the process performs an illegal or invalid action, often because of a bug. In this case, termination is usually forced by the operating system.

Common Causes

- executing an illegal instruction,
- referencing nonexistent memory,
- dividing by zero.

UNIX Behavior

In some systems such as UNIX, a process may request to handle certain errors itself. Then, instead of immediate termination, the operating system sends a **signal** to interrupt the process and let it respond.

5. Killed by Another Process

External Termination

A process may also terminate because another process explicitly asks the operating system to kill it.

Relevant Calls

- UNIX: `kill`
- Windows: `TerminateProcess`

Authorization Rule

The process requesting termination must have the necessary permission to kill the target process.

6. What Happens to Child Processes?

Possible Policy

Some operating systems immediately terminate all child processes when their parent process ends.

Important Distinction

This is **not** the default behavior in UNIX or Windows. In those systems, terminating one process does not automatically kill every process it created.

7. Key Contrast

Voluntary vs Involuntary

- **Voluntary termination:** the process decides to exit on its own.
- **Involuntary termination:** the operating system or another process forces termination.

8. Quick Review

Exam-Focused Points

- A process does not run forever; eventually it terminates.
- The four main reasons are normal exit, error exit, fatal error, and being killed by another process.
- `exit` in UNIX and `ExitProcess` in Windows are used for normal termination.
- Fatal errors usually come from illegal instructions, invalid memory access, or divide-by-zero errors.
- In UNIX, some errors can be handled through signals instead of immediate termination.
- `kill` in UNIX and `TerminateProcess` in Windows are used to terminate another process.
- UNIX and Windows do not automatically kill all child processes when a parent terminates.